ECUAȚIA DE GRADUL AL II-LEA

Proiect Testarea Sistemelor Software
T1 Testare Unitară în Python

BILIUȚĂ Elisa-Adelina [11], MARCOVICI Anamaria [21] și
NEDELESCU George-Eduard [31]

[1]Universitatea din București/ Facultatea de Matematică și Informatică/
Programul de studii universitare de licență Matematică-Informatică

Anul III, Seria 31, Grupa 311

[1]elisa-adelina.biliuta@s.unibuc.ro
[2]anamaria.marcovici@s.unibuc.ro
[3]george-eduard.nedelescu@s.unibuc.ro

Cuprins

1 Prezentarea problemei **2**

2 Testare funcțională **4**
2.1 Partiționarea în clase de echivalență (CE) 4
2.2 Analiza valorilor de frontieră (AVF) 7
2.3 Partiționarea în categorii (PC) 9

3 Testare structurală **13**
3.1 Graful de flux de control (CFG) 13
3.2 Analiza acoperirii (Coverage) 15
3.2.1 Acoperire la nivel de instrucțiuni 15
3.2.2 Acoperire la nivel de decizie 15
3.2.3 Legătura dintre cazurile de test și acoperirea codului 16
3.3 Acoperire la nivel de condiție 16
3.3.1 Condition/Decision Coverage 17
3.3.2 Multiple Condition Coverage 17
3.3.3 MC/DC 17
3.3.4 Path Coverage 18
3.3.5 LCSAJ 18
3.4 Complexitatea ciclomată (McCabe) 18
3.5 Circuite independente 19

4 Testare pe mutanți **20**
4.1 Definiție 20
4.1.1 Principii teoretice ale mutation testing 20
4.2 Operatori de mutație utilizați 20
4.2.1 Tipuri de mutanți 20
4.3 Implementarea clasei și a mutanților 21
4.4 Teste pentru implementarea originală și pentru mutanți 25
4.4.1 Strong vs Weak Mutation 26
4.5 Analiza mutanților 26
4.5.1 Condiții pentru detectarea mutanților 26
4.6 Rezultatele rulării testelor 27
4.6.1 Mutanți echivalenți 27
4.7 Concluzii 27

Capitolul 1

Prezentarea problemei

În acest capitol prezentăm problema principală pe care o vom supune testării, anume rezolvarea unei ecuații de gradul al II-lea de forma $ax^2 + bx + c = 0$, cu $a \in \mathbb{R}^*$ și $b, c \in \mathbb{R}$.

Programul implementat primește trei coeficienți reali (a, b, și c) și calculează soluțiile ecuației pe baza discriminantului $\Delta = b^2 - 4ac$. De asemenea, programul tratează cazurile de excepție, cum ar fi validarea tipului de date introdus și restricția matematică pentru care coeficientul $a \neq 0$.

Distingem mai jos cele trei cazuri posibile, precum și formulele de calcul ale soluțiilor ecuației, în funcție de semnul lui Δ:

$$x_{1,2} = \begin{cases} \frac{-b \pm \sqrt{\Delta}}{2a}, & \text{dacă } \Delta > 0 \\ -\frac{b}{2a}, & \text{dacă } \Delta = 0 \\ \frac{-b \pm i\sqrt{|\Delta|}}{2a}, & \text{dacă } \Delta < 0 \end{cases}.$$

În continuare, prezentăm implementarea clasei `EquationSolver` în limbajul Python.

```python
import math

class EquationSolver:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not all(isinstance(x, (int, float)) and not isinstance(x, bool) for x in [self.a, self.b, self.c]):
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 - 4 * self.a * self.c

        if delta < 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return x1, x2

        elif math.isclose(delta, 0.0, abs_tol=1e-9):
            return -self.b / (2 * self.a)

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return x1, x2
```

Secvență de cod 1.1: Fișierul `equation_solver.py` ilustrând clasa `EquationSolver`

Capitolul 2

Testare funcțională

În acest capitol detaliem strategiile de testare funcțională de tip *Black Box* aplicate asupra clasei `EquationSolver`. Testarea funcțională presupune generarea datelor de test pe baza specificațiilor programului, fără a ține cont de structura internă a codului.

2.1 Partiționarea în clase de echivalență (CE)

Domeniul de intrări este format din cei trei coeficienți (a, b, c), care trebuie să fie numere reale, iar $a \neq 0$. Pe baza verificării tipului de date introduse, a restricțiilor și a calculului discriminantului (Δ), am identificat un domeniu de ieșiri format din 5 tipuri distincte de rezultate. În continuare, vom analiza fiecare dintre aceste domenii și clasele de echivalență generate.

Domeniul de intrări conține:

- un număr real nenul a: determină clasele $A_1 = \mathbb{R}^*$, $A_2 = \{0\}$ și $A_3 = \mathbb{R}^c$;
- un număr real b: determină clasele $B_1 = \mathbb{R}$ și $B_2 = \mathbb{R}^c$;
- un număr real c: determină clasele $C_1 = \mathbb{R}$ și $C_2 = \mathbb{R}^c$.

Observație 1. Am notat cu $\mathbb{R}^c$ complementara mulțimii numerelor reale. Dacă `x` $\in \mathbb{R}^c$, atunci `x` are tipul de dată diferit de `int` și `float` în reprezentarea internă a limbajului Python (și nu are nici tip `bool`), lucru ce va genera erori la validarea tipului de date.

Domeniul de ieșiri constă în rezultatele posibile returnate de metoda `solve()` și poate fi explicitat astfel:

- mulțimea soluțiilor ecuației: determină clasele $S_1 = \mathbb{C} \times \mathbb{C}$, $S_2 = \mathbb{R}$, $S_3 = \mathbb{R} \times \mathbb{R}$;
- stările de eroare: determină clasele: $E_1 =$ `ValueError` (excepție aruncată când cel puțin un coeficient aparține mulțimii $\mathbb{R}^c$) și $E_2 =$ `ValueError` (excepție aruncată pentru restricția matematică $a \neq 0$).

Observație 2. Clasa S_1 presupune un tuplu de două rădăcini complexe conjugate (când $\Delta < 0$), clasa S_2 presupune o singură rădăcină reală (rădăcină dublă, când $\Delta = 0$), iar clasa S_3 presupune un tuplu de două rădăcini reale distincte (când $\Delta > 0$).

Pe baza domeniului de ieșiri, folosind principiul metodei „reverse engineering” (de la ieșiri spre intrări) și ținând cont de fluxul de validare care previne evaluările multiple (o excepție aruncată oprește execuția), putem partiționa domeniul de intrări în trei clase disjuncte:

1. **Intrări care declanșează eroarea E_1 (Validarea tipului):** Execuția se oprește prematur dacă cel puțin un coeficient nu este număr real. Aceasta generează prima clasă de echivalență globală:

$$C_{E_1} = \{(a, b, c) \mid a \in A_3 \vee b \in B_2 \vee c \in C_2\}$$

2. **Intrări care nu declanșează eroarea E_1, dar care declanșează eroarea E_2 (Restricția matematică):** Acest caz presupune că toate datele sunt numere reale ($a \in A_1 \cup A_2$, $b \in B_1$, $c \in C_1$), dar coeficientul dominant este zero ($a \in A_2$). Obținem astfel a doua clasă de echivalență globală:

$$C_{E_2} = A_2 \times B_1 \times C_1 = \{(0, b, c) \mid b, c \in \mathbb{R}\}$$

3. **Intrări care nu declanșează nicio eroare:** Acest caz presupune date valide: avem coeficienți reali și $a \neq 0$, adică domeniul $A_1 \times B_1 \times C_1$. Facem discuție după calculul discriminantului Δ, care subpartiționează domeniul valid în cele 3 clase corespunzătoare soluțiilor:

$$C_{S_1} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac < 0\}$$
$$C_{S_2} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac = 0\}$$
$$C_{S_3} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac > 0\}$$

Pe baza partiționării anterioare, am selectat câte un caz de testare reprezentativ pentru fiecare clasă de echivalență. Seturile de date și rezultatele așteptate sunt sintetizate în tabelul de mai jos:

ID Test	Clasa	a	b	c	Δ	Rezultat așteptat
TC_CE_01	C_{E_1}	`"text"`	2	1	N/A	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_CE_02	C_{E_2}	0	2	1	N/A	$\boldsymbol{E_2}$ (Excepție: a nu poate fi 0)
TC_CE_03	C_{S_1}	1	1	1	-3	$\boldsymbol{S_1}$ (Două rădăcini complexe)
TC_CE_04	C_{S_2}	1	2	1	0	$\boldsymbol{S_2}$ (Rădăcină reală dublă)
TC_CE_05	C_{S_3}	1	5	6	1	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)

Tabela 2.1: Cazuri de testare pentru metoda CE

Observație 3. Se remarcă faptul că pentru clasele de excepție C_{E_1} și C_{E_2}, discriminantul Δ nu este calculat, deoarece execuția este întreruptă înainte de a ajunge la evaluarea formulei matematice. Această afirmație nu încalcă principiile strategiei *Black Box*.

Pentru a valida practic scenariile teoretice definite anterior, în secvența de cod următoare prezentăm implementarea automatizată a cazurilor de testare extrase din Tabelul 2.1, utilizând frameworkul `unittest` din limbajul Python. Deoarece urmează se efectuează calcule analitice pe valori reale, verificările folosesc extensiv mecanismul `assertAlmostEqual` pentru a preveni eșuarea testelor pe fondul erorilor inevitabile de trunchiere și rotunjire în virgulă mobilă.

```python
class TestEquationSolverCE(unittest.TestCase):
    # TC_CE_01: Clasa CE1 - Validarea tipului de date (cel putin un coeficient nu este numar real); vom trata fenomenul defect masking
    def test_tc_ce_01_a_invalid(self):
        solver = EquationSolver("text", 2, 1)
        with self.assertRaises(ValueError):
            solver.solve()

    def test_tc_ce_01_b_invalid(self):
        solver = EquationSolver(1, "text", 1)
        with self.assertRaises(ValueError):
            solver.solve()

    def test_tc_ce_01_c_invalid(self):
        solver = EquationSolver(1, 2, "text")
        with self.assertRaises(ValueError):
            solver.solve()

    # TC_CE_02: Clasa CE2 - Restrictia matematica (a != 0)
    def test_tc_ce_02_a_zero(self):
        solver = EquationSolver(0, 2, 1)
        with self.assertRaises(ValueError):
            solver.solve()

    # TC_CE_03: Clasa CS1 - Delta < 0 (Doua radacini complexe conjugate)
    def test_tc_ce_03_delta_negativ(self):
        solver = EquationSolver(1, 1, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, complex)
        self.assertIsInstance(x2, complex)

        self.assertAlmostEqual(x1.real, -0.5)
        self.assertAlmostEqual(x2.real, -0.5)

        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.866025, places=5)
        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.866025, places=5)

    # TC_CE_04: Clasa CS2 - Delta = 0 (O radacina reala dubla)
    def test_tc_ce_04_delta_zero(self):
        solver = EquationSolver(1, 2, 1)
        x = solver.solve()

        self.assertIsInstance(x, float)
        self.assertAlmostEqual(x, -1.0)

    # TC_CE_05: Clasa CS3 - Delta > 0 (Doua radacini reale distincte)
    def test_tc_ce_05_delta_pozitiv(self):
        solver = EquationSolver(1, 5, 6)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -2.0, places=5)
        self.assertAlmostEqual(min(x1, x2), -3.0, places=5)
```

Secvență de cod 2.1: Fișierul `test_equation_solver.py` ilustrând clasa `TestEquationSolverCE`

2.2 Analiza valorilor de frontieră (AVF)

Conform principiilor de bază ale metodei AVF, pentru fiecare frontieră identificată trebuie testată atât valoarea exactă a limitei, cât și valorile imediat adiacente (din interiorul și din afara clasei valide). Pentru problema ecuației de gradul al II-lea, deși prelucrăm un domeniu continuu, reprezentat de mulțime $\mathbb{R}$, distingem clar două frontiere decizionale impuse de fluxul de control al programului: trecerea dintre valid și invalid ($a = 0$) și punctul critic dintre rădăcinile reale și cele complexe ($\Delta = 0$). Să detaliem:

- **Frontiera coeficientului dominant (a):** Restricția matematică impune $a \neq 0$. Conform teoriei AVF, setul de teste trebuie să evalueze cazul-limită ($a = 0$) și valorile imediat adiacente acestuia ($a \to 0^+$ și $a \to 0^-$). Întrucât testul pentru valoarea $a = 0$ a fost deja executat sub umbrela metodei CE (reprezentând clasa de excepție C_{E_2}), analiza de frontieră va fi completată prin injectarea unor valori arbitrar de mici, notate cu $\pm\epsilon$ (unde vom alege $\epsilon = 0.001$).
- **Frontiera discriminantului (Δ):** Valoare critică care produce separarea rădăcinilor reale de cele complexe este $\Delta = 0$. Cazul-limită ($\Delta = 0$) reprezintă însăși clasa C_{S_2} din metoda CE. Tehnica AVF impune completarea cu testarea vecinătăților $\Delta \to 0^+$ și $\Delta \to 0^-$, stări care pot fi forțate prin variația fină a coeficientului b în jurul valorii 2, menținând $a = 1$ și $c = 1$, de exemplu.

Pentru a oferi o imagine completă a acoperirii la frontieră, Tabelul 2.2 sintetizează întregul spectru de teste AVF, înglobând atât valorile pentru frontieră (moștenite din partiționarea de echivalență), cât și cazurile pentru vecinătățile acesteia.

ID Test	Frontieră	a	b	c	Δ	Rezultat așteptat
TC_CE_02	$a = 0$	0	2	1	N/A	$\boldsymbol{E_2}$ (Excepție: a nu poate fi 0)
TC_AVF_01	$a \to 0^-$	-0.001	2	1	4.004	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)
TC_AVF_02	$a \to 0^+$	0.001	2	1	3.996	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)
TC_CE_04	$\Delta = 0$	1	2	1	0	$\boldsymbol{S_2}$ (Rădăcină reală dublă)
TC_AVF_03	$\Delta \to 0^-$	1	1.999	1	≈ -0.004	$\boldsymbol{S_1}$ (Două rădăcini complexe)
TC_AVF_04	$\Delta \to 0^+$	1	2.001	1	≈ 0.004	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)

Tabela 2.2: Cazuri de testare pentru metoda AVF

Observație 4. Spre deosebire de mulțimile de tip `int`, unde valoarea adiacentă este precis determinabilă ($+1$ sau -1), pe un domeniu continuu de tip `float` conceptul de „valoare imediat adiacentă” este mărginit doar de limitele de reprezentare arhitecturală a mașinii. Din perspectiva strategiei *Black Box*, o toleranță mică $\epsilon = 10^{-3}$ este optimă și suficientă pentru a garanta evaluarea ramurilor decizionale la extremitățile lor critice.

Implementarea automatizată a scenariilor AVF este redată mai jos.

```python
class TestEquationSolverAVF(unittest.TestCase):
    # Testele pentru a=0 si Delta=0 au fost deja implementate in clasa TestEquationSolverCE (test_tc_ce_02 si test_tc_ce_04)

    # TC_AVF_01: Limita stanga pentru coeficientul a (a -> 0-)
    def test_tc_avf_01_a_limita_negativa(self):
        solver = EquationSolver(-0.001, 2, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), 2000.49988, places=3)
        self.assertAlmostEqual(min(x1, x2), -0.49987, places=3)

    # TC_AVF_02: Limita dreapta pentru coeficientul a (a -> 0+)
    def test_tc_avf_02_a_limita_pozitiva(self):
        solver = EquationSolver(0.001, 2, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -0.50012, places=3)
        self.assertAlmostEqual(min(x1, x2), -1999.49987, places=3)

    # TC_AVF_03: Limita stanga pentru Delta (Delta -> 0-)
    def test_tc_avf_03_delta_limita_negativa(self):
        solver = EquationSolver(1, 1.999, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, complex)
        self.assertIsInstance(x2, complex)

        self.assertAlmostEqual(x1.real, -0.9995, places=3)
        self.assertAlmostEqual(x2.real, -0.9995, places=3)

        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.03161, places=3)
        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.03161, places=3)

    # TC_AVF_04: Limita dreapta pentru Delta (Delta -> 0+)
    def test_tc_avf_04_delta_limita_pozitiva(self):
        solver = EquationSolver(1, 2.001, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -0.96887, places=3)
        self.assertAlmostEqual(min(x1, x2), -1.03213, places=1)
```

Secvență de cod 2.2: Fișierul `test_equation_solver.py` ilustrând clasa `TestEquationSolverAVF`

2.3 Partiționarea în categorii (PC)

Metoda partiționării în categorii extinde tehnicile anterioare, având ca scop generarea unui set de date de test care acoperă exhaustiv funcționalitatea sistemului și maximizează posibilitatea descoperirii erorilor. Această abordare structurată descompune specificația în unități testabile, identifică parametrii, categoriile acestora și partiționează fiecare categorie în alternative. Pentru a preveni explozia combinatorică inerentă și a elimina cazurile care nu au sens, se introduc constrângeri logice (condiții de selecție) asupra alternativelor.

Urmărind pașii metodologici, aplicăm metoda PC pentru clasa `EquationSolver`:

Pasul 1: Descompunerea specificației în unități

Sistemul prezintă o singură unitate funcțională testabilă izolată: metoda `solve()`.

Pasul 2: Identificarea parametrilor și a condițiilor de mediu

Parametrii de intrare care influențează direct comportamentul sunt coeficienții a, b, c. Condiția de mediu relevantă (starea internă rezultată în urma procesării datelor valide) este discriminantul Δ.

Pasul 3: Găsirea categoriilor

Proprietățile majore care definesc și separă fluxurile de execuție ale programului sunt:

- $\boldsymbol{C_1}$ *(Validitatea tipului de date)*: verifică apartenența $a, b, c \in \mathbb{R}$;
- $\boldsymbol{C_2}$ *(Valoarea coeficientului dominant)*: impune restricția $a \neq 0$;
- $\boldsymbol{C_3}$ *(Natura rădăcinilor)*: dictată de semnul lui Δ.

Pașii 4 și 5: Partiționarea în alternative și scrierea specificației de testare

Pentru a eficientiza testarea și a evita generarea de combinații redundante, asociem alternativelor constrângeri. Proprietatea `[error]` va opri combinarea ulterioară, deoarece declanșează întreruperea execuției programului. Pentru a asigura o testare robustă, alternativele integrează și valorile de frontieră identificate anterior. Să detaliem:

- **Parametrul a:**
  1. $a \in \mathbb{R}^c$ `[error]`;
  2. $a = 0$ `[error]`;
  3. $a \to 0^-$ `[property Valid_A]`;
  4. $a \to 0^+$ `[property Valid_A]`;
  5. $a \in \mathbb{R} \setminus [-\epsilon, \epsilon]$ `[property Valid_A]`.
- **Parametrul b:**
  1. $b \in \mathbb{R}^c$ `[error]`;
  2. $b \in \mathbb{R}$ `[property Valid_B]`.
- **Parametrul c:**
  1. $c \in \mathbb{R}^c$ `[error]`;
  2. $c \in \mathbb{R}$ `[property Valid_C]`.
- **Semnul lui Δ:**
  1. $\Delta \ll 0$ `[if Valid_A and Valid_B and Valid_C]`;
  2. $\Delta \to 0^-$ `[if Valid_A and Valid_B and Valid_C]`;
  3. $\Delta = 0$ `[if Valid_A and Valid_B and Valid_C]`;
  4. $\Delta \to 0^+$ `[if Valid_A and Valid_B and Valid_C]`;
  5. $\Delta \gg 0$ `[if Valid_A and Valid_B and Valid_C]`.

Pașii 6 și 7: Crearea cazurilor și a datelor de test

În absența constrângerilor, simplul produs cartezian al alternativelor ar fi generat $5 \times 2 \times 2 \times 5 = 100$ de combinații. Aplicând condițiile de selecție, excludem evaluarea lui Δ pentru intrările care declanșează erori. De asemenea, pentru optimizare, într-un caz de testare la limita frontierelor, testăm doar câte o singură frontieră, reducând astfel spațiul teoretic la 11 scenarii de testare distincte, care pot fi observate în Tabelul 2.3.

Acest set de teste este optim, deoarece cuprinde:

- **4 cazuri de excepție**: acoperă validarea datelor (3 teste) și restricția matematică esențială (1 test);
- **2 cazuri pentru frontiera intrărilor**: testează stabilitatea programului când a tinde la 0 din ambele direcții;
- **2 cazuri pentru frontiera stărilor interne**: testează sensibilitatea decizională la granița dintre soluțiile reale și cele complexe;
- **3 cazuri de clasă normală**: validează comportamentul standard pentru cele trei tipuri de rădăcini.

ID Test	Combinații	a	b	c	Rezultat așteptat
TC_PC_01	a_1	True	2	1	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_PC_02	$a_5 \wedge b_1$	1	None	1	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_PC_03	$a_5 \wedge b_2 \wedge c_1$	1	2	[]	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_PC_04	$a_2 \wedge b_2 \wedge c_2$	0	2	1	$\boldsymbol{E_2}$ (Excepție: a nu poate fi 0)
TC_PC_05	$a_3 \wedge b_2 \wedge c_2 \wedge \Delta_5$	-0.001	2	1	$\boldsymbol{S_3}$ ($a \to 0^-$; $\Delta > 0$)
TC_PC_06	$a_4 \wedge b_2 \wedge c_2 \wedge \Delta_5$	0.001	2	1	$\boldsymbol{S_3}$ ($a \to 0^+$; $\Delta > 0$)
TC_PC_07	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_2$	1	1.999	1	$\boldsymbol{S_1}$ (a normal; $\Delta \to 0^-$)
TC_PC_08	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_3$	1	2	1	$\boldsymbol{S_2}$ (a normal; $\Delta = 0$)
TC_PC_09	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_4$	1	2.001	1	$\boldsymbol{S_3}$ (a normal; $\Delta \to 0^+$)
TC_PC_10	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_1$	1	1	1	$\boldsymbol{S_1}$ (valori normale; $\Delta < 0$)
TC_PC_11	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_5$	1	5	6	$\boldsymbol{S_3}$ (valori normale; $\Delta > 0$)

Tabela 2.3: Cazuri de testare pentru metoda PC

Pentru a încheia secțiunea, prezentăm implementarea automatizată a scenariilor sumarizate mai sus în fragmentul de cod următor.

```python
class TestEquationSolverPC(unittest.TestCase):
    # TC_PC_01 -> TC_PC_03: Erori de validare a tipului de date pentru coeficientii a, b, c
    def test_tc_pc_01_a_invalid(self):
        solver = EquationSolver(True, 2, 1)
        with self.assertRaises(ValueError):
            solver.solve()

    def test_tc_pc_02_b_invalid(self):
        solver = EquationSolver(1, None, 1)
        with self.assertRaises(ValueError):
            solver.solve()

```

```python
    def test_tc_pc_03_c_invalid(self):
        solver = EquationSolver(1, 2, [])
        with self.assertRaises(ValueError):
            solver.solve()

    # TC_PC_04: Valoare invalida matematic (a = 0)
    def test_tc_pc_04_a_zero(self):
        solver = EquationSolver(0, 2, 1)
        with self.assertRaises(ValueError):
            solver.solve()

    # TC_PC_05: Limita stanga pentru coeficientul a (a -> 0-)
    def test_tc_pc_05_a_limita_negativa(self):
        solver = EquationSolver(-0.001, 2, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), 2000.49988, places=3)
        self.assertAlmostEqual(min(x1, x2), -0.49987, places=3)

    # TC_PC_06: Limita dreapta pentru coeficientul a (a -> 0+)
    def test_tc_pc_06_a_limita_pozitiva(self):
        solver = EquationSolver(0.001, 2, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -0.50012, places=3)
        self.assertAlmostEqual(min(x1, x2), -1999.49987, places=3)

    # TC_PC_07: Limita stanga pentru Delta (Delta -> 0-)
    def test_tc_pc_07_delta_limita_negativa(self):
        solver = EquationSolver(1, 1.999, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, complex)
        self.assertIsInstance(x2, complex)

        self.assertAlmostEqual(x1.real, -0.9995, places=3)
        self.assertAlmostEqual(x2.real, -0.9995, places=3)

        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.03161, places=3)
        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.03161, places=3)

    # TC_PC_08: Delta = 0 (O radacina reala dubla)
    def test_tc_pc_08_delta_zero(self):
        solver = EquationSolver(1, 2, 1)
        x = solver.solve()

        self.assertIsInstance(x, float)
        self.assertEqual(x, -1.0)

    # TC_PC_09: Limita dreapta pentru Delta (Delta -> 0+)
    def test_tc_pc_09_delta_limita_pozitiva(self):
        solver = EquationSolver(1, 2.001, 1)
```

```python
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -0.96887, places=3)
        self.assertAlmostEqual(min(x1, x2), -1.03213, places=3)

    # TC_PC_10: Delta < 0 (Doua radacini complexe)
    def test_tc_pc_10_delta_negativ(self):
        solver = EquationSolver(1, 1, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, complex)
        self.assertIsInstance(x2, complex)

        self.assertAlmostEqual(x1.real, -0.5)
        self.assertAlmostEqual(x2.real, -0.5)

        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.866025, places=5)
        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.866025, places=5)

    # TC_PC_11: Delta > 0 (Doua radacini reale distincte)
    def test_tc_pc_11_delta_pozitiv(self):
        solver = EquationSolver(1, 5, 6)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -2.0, places=5)
        self.assertAlmostEqual(min(x1, x2), -3.0, places=5)
```

Secvență de cod 2.3: Fișierul `test_equation_solver.py` ilustrând clasa `TestEquationSolverPC`

Rezultatul obținut la rularea tuturor testelor din acest capitol este prezentat mai jos:

```
"D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\.venv\Scripts\
    python.exe" "D:/SOFTURI/PyCharm 2025.3.3/plugins/python-ce/helpers/
    pycharm/_jb_unittest_runner.py" --path "D:\FMI\ANUL III\TESTAREA
    SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py"
Testing started at 17:02 ...
Launching unittests with arguments python -m unittest D:\FMI\ANUL III\
    TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py --quiet
     in D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS

Ran 22 tests in 0.024s

OK

Process finished with exit code 0
```

Capitolul 3

Testare structurală

Testarea structurală (White-Box Testing) reprezintă o metodă de testare care analizează structura internă a codului sursă. Aceasta urmărește fluxul de control, ramificațiile și condițiile logice ale programului, având ca scop executarea tuturor instrucțiunilor și deciziilor cel puțin o dată în timpul testării.

În cadrul acestui capitol se realizează analiza structurală a metodei `solve()` din clasa `EquationSolver`, prin construirea grafului de flux de control (Control Flow Graph), determinarea complexității ciclomate, identificarea drumurilor independente și evaluarea gradului de acoperire a codului pe baza cazurilor de test definite anterior.

3.1 Graful de flux de control (CFG)

Graful de flux de control (CFG) reprezintă o modelare grafică a modului în care instrucțiunile unui program sunt executate. Nodurile grafului corespund blocurilor de instrucțiuni, iar muchiile indică tranzițiile dintre acestea.

Analiza CFG permite evidențierea ramificațiilor logice și identificarea drumurilor de execuție ale programului.

Pentru metoda `solve()`, graful de flux de control surprinde etapele principale ale algoritmului: validarea coeficienților, verificarea condiției $a \neq 0$, calculul discriminantului și ramificarea execuției în funcție de valoarea acestuia.

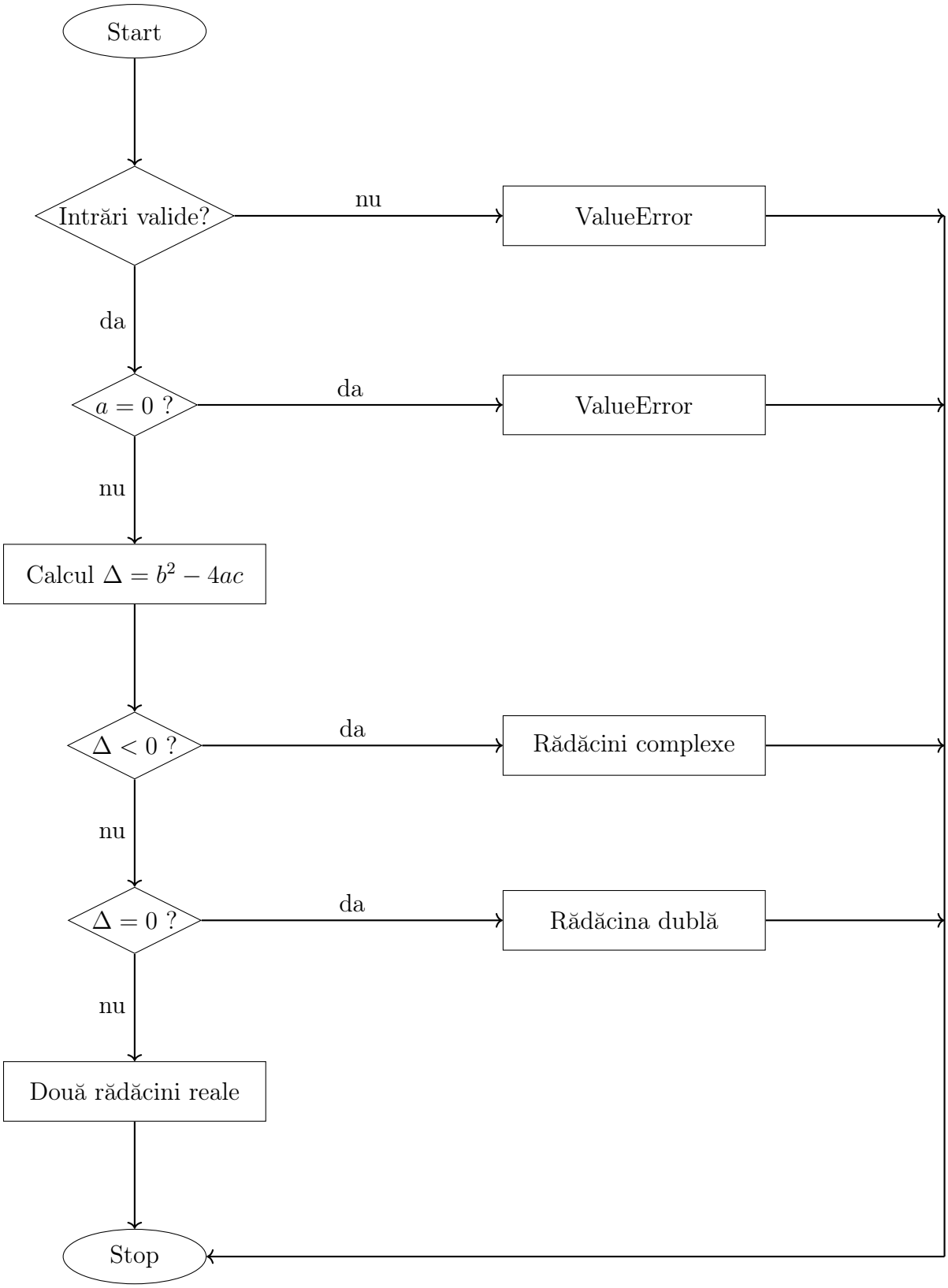

Figura 3.1: Graful de flux de control pentru metoda `solve()`

Graful evidențiază succesiunea pașilor de execuție. Execuția începe în nodul *Start*, unde sunt validate intrările. Dacă cel puțin un coeficient nu este numeric, execuția este întreruptă prin generarea unei excepții.

În cazul în care datele sunt valide, se verifică valoarea coeficientului a. Dacă $a = 0$, execuția este oprită, deoarece ecuația nu mai este de gradul al doilea. În caz contrar, se calculează discriminantul $\Delta = b^2 - 4ac$.

În funcție de valoarea discriminantului, execuția se ramifică în trei situații:

- $\Delta < 0$ – două rădăcini complexe;
- $\Delta = 0$ – o rădăcină reală dublă;
- $\Delta > 0$ – două rădăcini reale distincte.

După determinarea rezultatului, execuția ajunge în nodul final *Stop*.

3.2 Analiza acoperirii (Coverage)

Analiza acoperirii codului urmărește în ce măsură cazurile de test exercită diferitele componente ale programului. Scopul este verificarea faptului că toate instrucțiunile și toate ramurile logice sunt executate cel puțin o dată.

Acoperirea la nivel de decizie este echivalentă cu acoperirea la nivel de ramură (branch coverage). Pentru metoda `solve()`, sunt analizate două tipuri principale de acoperire: acoperirea la nivel de instrucțiuni și acoperirea la nivel de decizie.

3.2.1 Acoperire la nivel de instrucțiuni

Acoperirea la nivel de instrucțiuni (Statement Coverage) presupune executarea fiecărei instrucțiuni cel puțin o dată.

În cazul metodei `solve()`, acest lucru implică testarea tuturor situațiilor posibile:

- intrări invalide (coeficienți nenumerici);
- cazul $a = 0$;
- $\Delta < 0$;
- $\Delta = 0$;
- $\Delta > 0$.

3.2.2 Acoperire la nivel de decizie

Acoperirea la nivel de decizie presupune evaluarea fiecărei condiții logice atât cu valoarea *true*, cât și *false*.

Pentru metoda analizată, aceasta implică:

- verificarea validității intrărilor;
- verificarea condiției $a = 0$;
- evaluarea celor trei cazuri determinate de discriminant.

Prin cazurile de test definite în Capitolul 2, toate ramurile logice sunt exercitate, asigurând acoperire completă la nivel de instrucțiuni și decizie.

3.2.3 Legătura dintre cazurile de test și acoperirea codului

Pentru a demonstra acoperirea completă, corelăm cazurile de test definite cu ramurile din CFG.

- **Validarea tipului de date (E_1)**
  acoperită de: `TC_CE_01`, `TC_PC_01`, `TC_PC_02`, `TC_PC_03`
- **Restricția $a = 0$ (E_2)**
  acoperită de: `TC_CE_02`, `TC_PC_04`
- $\Delta < 0$
  acoperit de: `TC_CE_03`, `TC_PC_10`, `TC_AVF_03`
- $\Delta = 0$
  acoperit de: `TC_CE_04`, `TC_PC_08`
- $\Delta > 0$
  acoperit de: `TC_CE_05`, `TC_PC_11`, `TC_AVF_01`, `TC_AVF_02`, `TC_AVF_04`

Prin această corespondență se confirmă că toate ramurile logice sunt testate, iar acoperirea codului este completă.

3.3 Acoperire la nivel de condiție

Acoperirea la nivel de condiție (Condition Coverage) urmărește evaluarea fiecărei condiții elementare dintr-o expresie logică atât cu valoarea adevărat, cât și fals.

Scopul acestei metrici este de a verifica toate componentele unei condiții, nu doar rezultatul final al deciziei.

În cazul metodei `solve()`, condițiile sunt în mare parte simple (nu există expresii logice compuse explicite), astfel încât acoperirea la nivel de condiție coincide în mare măsură cu acoperirea la nivel de decizie.

Condițiile identificate în cod sunt:

- validarea tipului coeficienților: `not all(...)`
- verificarea coeficientului dominant: `a == 0`
- evaluarea discriminantului: `delta < 0`
- verificarea cazului limită: `math.isclose(delta, 0)`

Pentru a demonstra acoperirea la nivel de condiție, analizăm pentru fiecare condiție cazurile în care aceasta este evaluată atât ca adevărat, cât și fals.

Condiție	Evaluare True	Evaluare False
`not all(...)`	TC_CE_01, TC_PC_01–03	TC_CE_02–05, TC_PC_05–11
`a == 0`	TC_CE_02, TC_PC_04	toate testele valide
`delta < 0`	TC_CE_03, TC_PC_10, TC_AVF_03	TC_CE_04–05, TC_PC_08–09, TC_AVF_01–02–04
`isclose(delta, 0)`	TC_CE_04, TC_PC_08	cazurile cu $\Delta < 0$ și $\Delta > 0$

Tabela 3.1: Acoperirea condițiilor pentru metoda `solve()`

Se observă că fiecare condiție este evaluată atât cu valoarea adevărat, cât și cu valoarea fals, ceea ce confirmă obținerea unei acoperiri complete la nivel de condiție.

3.3.1 Condition/Decision Coverage

Acoperirea condiție/decizie (Condition/Decision Coverage) combină acoperirea la nivel de condiție cu acoperirea la nivel de decizie, impunând atât evaluarea condițiilor individuale, cât și a rezultatului global al deciziei.

Această metrică oferă o analiză mai riguroasă decât acoperirea la nivel de decizie simplă.

Pentru metoda `solve()`, această acoperire este satisfăcută, deoarece fiecare condiție este evaluată pentru ambele rezultate posibile, iar fiecare ramură decizională este parcursă de cel puțin un caz de test.

3.3.2 Multiple Condition Coverage

Acoperirea la nivel de condiții multiple (Multiple Condition Coverage) presupune testarea tuturor combinațiilor posibile de valori de adevăr ale condițiilor individuale dintr-o expresie logică.

Această metodă este mai strictă decât acoperirea condiție/decizie, însă poate conduce la un număr foarte mare de cazuri de test.

În cazul metodei `solve()`, nu există condiții compuse explicite (de tip `c1 && c2` sau `c1 || c2`), astfel încât această metrică nu aduce informații suplimentare relevante

3.3.3 MC/DC

Acoperirea MC/DC (Modified Condition/Decision Coverage) impune ca fiecare condiție individuală să influențeze în mod independent rezultatul deciziei.

Aceasta este utilizată în special în sisteme critice, unde este necesară o analiză detaliată a influenței fiecărei condiții.

Pentru metoda analizată, lipsa condițiilor compuse face ca aplicarea acestei metode să nu fie necesară.

3.3.4 Path Coverage

Testarea la nivel de cale (Path Coverage) presupune generarea unor cazuri de test astfel încât fiecare cale posibilă din graful de flux de control să fie parcursă.

În cazul metodei `solve()`, căile de execuție sunt finite și corespund ramurilor principale:

- eroare de tip (E_1)
- eroare pentru $a = 0$ (E_2)
- $\Delta < 0$
- $\Delta = 0$
- $\Delta > 0$

Acestea sunt deja acoperite prin cazurile de test definite în Capitolul 2, ceea ce asigură o acoperire completă la nivel de cale.

3.3.5 LCSAJ

Acoperirea LCSAJ (Linear Code Sequence and Jump) analizează secvențele liniare de cod urmate de salturi ale fluxului de execuție.

Această metodă oferă o granularitate mai mare decât acoperirea la nivel de decizie.

Pentru metoda `solve()`, structura simplă a codului și numărul redus de ramuri nu justifică aplicarea acestei metode, deoarece nu aduce beneficii suplimentare semnificative.

3.4 Complexitatea ciclomată (McCabe)

Complexitatea ciclomatică reprezintă o măsură a complexității logice a unui program și indică numărul minim de drumuri independente care trebuie testate pentru a asigura acoperirea completă a codului. Aceasta a fost introdusă de McCabe și se calculează folosind relația:

$$V(G) = E - N + 2P$$

unde:

- E reprezintă numărul de muchii ale grafului de flux de control;
- N reprezintă numărul de noduri ale grafului;
- P reprezintă numărul de componente conexe ale grafului (în cazul analizat, $P = 1$).

Complexitatea ciclomată poate fi determinată și prin numărul de decizii din program, folosind relația:

$$V(G) = D + 1$$

unde D reprezintă numărul de decizii.

În cazul metodei `solve()`, se identifică trei decizii principale:

- verificarea validității coeficienților;
- verificarea condiției $a = 0$;
- evaluarea discriminantului Δ.

Astfel, complexitatea ciclomată este:

$$V(G) = 3 + 1 = 4$$

Această valoare indică faptul că există patru drumuri independente care trebuie testate pentru a asigura acoperirea completă a metodei.

Analiza realizată se bazează pe tehnica *Basis Path Testing*, care utilizează complexitatea ciclomată pentru a determina setul minim de drumuri independente necesare testării complete.

3.5 Circuite independente

Circuitele independente (sau drumurile independente) reprezintă trasee distincte în graful de flux de control, fiecare dintre acestea introducând cel puțin o muchie nouă care nu a mai fost parcursă anterior.

Numărul de circuite independente este determinat de complexitatea ciclomată a programului. În cazul metodei `solve()`, complexitatea ciclomată este $V(G) = 4$, ceea ce înseamnă că există patru drumuri independente ce trebuie testate pentru a asigura acoperirea completă a codului. Aceste drumuri sunt:

- **Drumul 1**: validarea intrărilor eșuează (coeficienții nu sunt valori numerice valide), iar execuția se oprește prin generarea unei excepții;
- **Drumul 2**: intrările sunt valide, coeficientul $a \neq 0$, iar discriminantul $\Delta < 0$, rezultând două rădăcini complexe;
- **Drumul 3**: intrările sunt valide, coeficientul $a \neq 0$, iar discriminantul $\Delta = 0$, rezultând o rădăcină reală dublă;
- **Drumul 4**: intrările sunt valide, coeficientul $a \neq 0$, iar discriminantul $\Delta > 0$, rezultând două rădăcini reale distincte.

Parcurgerea acestor drumuri prin intermediul unor cazuri de test adecvate asigură exercitarea tuturor ramurilor logice ale metodei `solve()` și, implicit, acoperirea completă a codului.

Analiza structurală realizată evidențiază faptul că metoda `solve()` prezintă o structură logică simplă, cu un număr redus de decizii. Setul de teste definit acoperă toate ramurile de execuție identificate, asigurând atât acoperire completă la nivel de instrucțiuni, cât și la nivel de decizie. Complexitatea ciclomată redusă confirmă faptul că programul este ușor de testat și de întreținut.

Capitolul 4

Testare pe mutanți

4.1 Definiție

Testarea pe mutanți (Mutation Testing) este o tehnică de testare structurală utilizată pentru a evalua calitatea și eficiența unui set de teste. Ideea principală constă în introducerea unor modificări mici și controlate în codul sursă, numite *mutanți*, care simulează erori de programare plauzibile.

După generarea mutanților, se rulează setul de teste existent asupra fiecărei variante modificate a programului. Dacă testele detectează modificarea și eșuează, mutantul este considerat *ucis* (*killed*). Dacă testele continuă să treacă, mutantul *supraviețuiește* (*survives*), ceea ce indică o slăbiciune a setului de teste.

Prin urmare, testarea pe mutanți oferă o măsură mai riguroasă a calității testelor decât simpla analiză de acoperire, deoarece verifică nu doar dacă o porțiune de cod a fost executată, ci și dacă testele sunt suficient de precise pentru a detecta comportamente incorecte.

4.1.1 Principii teoretice ale mutation testing

Mutation testing se bazează pe două ipoteze fundamentale:

- **Competent Programmer Hypothesis (CPH)**: se presupune că un programator scrie un program apropiat de unul corect, iar erorile sunt mici deviații.
- **Coupling Effect**: testele care detectează erori simple (mutanți de ordin 1) sunt, în general, suficiente pentru a detecta și erori mai complexe.

4.2 Operatori de mutație utilizați

Pentru evaluarea clasei `EquationSolver`, au fost selectate mai multe categorii de operatori de mutație, care vizează punctele critice ale algoritmului: calculul discriminantului, condițiile de ramificare, validarea datelor de intrare și valorile returnate de metodă.

4.2.1 Tipuri de mutanți

Mutanții pot fi clasificați în funcție de numărul de modificări aplicate:

- **Mutanți de ordin 1 (first-order mutants)**: obținuți printr-o singură modificare.

- **Mutanți de ordin superior (higher-order mutants)**: obținuți prin aplicarea mai multor modificări succesive.

În practică, se utilizează în principal mutanții de ordin 1, deoarece numărul mutanților crește foarte rapid pentru ordine mai mari.

Tabela 4.1: Clasificarea mutanților pentru metoda `solve()`

Tip mutant	Cod original	Cod mutant	Explicație
AOR	`b**2 - 4*a*c`	`b**2 + 4*a*c`	Modifică formula de calcul a discriminantului.
ROR	`delta < 0`	`delta <= 0`	Afectează decizia de la frontiera $\Delta = 0$.
LCR	`all(...)`	`any(...)`	Slăbește validarea logică a coeficienților introduși.
STD-1	`return (x1, x2)`	`return None`	Modifică rezultatul returnat pe ramura cu rădăcini complexe.
STD-2	`return -b/(2*a)`	`return -b/a`	Modifică formula pentru cazul $\Delta = 0$.

4.3 Implementarea clasei și a mutanților

În continuare este prezentată implementarea clasei originale `EquationSolver`, precum și a mutanților utilizați în analiză.

```python
import math
import unittest

class EquationSolver:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not all(isinstance(x, (int, float)) for x in [self.a, self.b, self.c]):
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 - 4 * self.a * self.c

        if delta < 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return (x1, x2)

        elif delta == 0:
```

```
            return -self.b / (2 * self.a)

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return (x1, x2)


class EquationSolver_Mutant_AOR:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not all(isinstance(x, (int, float)) for x in [self.a, self.b, self.c]):
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 + 4 * self.a * self.c

        if delta < 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return (x1, x2)

        elif delta == 0:
            return -self.b / (2 * self.a)

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return (x1, x2)


class EquationSolver_Mutant_ROR:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not all(isinstance(x, (int, float)) for x in [self.a, self.b, self.c]):
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 - 4 * self.a * self.c

```

```
        if delta <= 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return (x1, x2)

        elif delta == 0:
            return -self.b / (2 * self.a)

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return (x1, x2)


class EquationSolver_Mutant_LCR:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not any(isinstance(x, (int, float)) for x in [self.a, self.b, self.c]):
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 - 4 * self.a * self.c

        if delta < 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return (x1, x2)

        elif delta == 0:
            return -self.b / (2 * self.a)

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return (x1, x2)


class EquationSolver_Mutant_STD1:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not all(isinstance(x, (int, float)) for x in [self.a, self.b, self.c]):
```

```
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 - 4 * self.a * self.c

        if delta < 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return None

        elif delta == 0:
            return -self.b / (2 * self.a)

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return (x1, x2)


class EquationSolver_Mutant_STD2:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not all(isinstance(x, (int, float)) for x in [self.a, self.b, self.c]):
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 - 4 * self.a * self.c

        if delta < 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return (x1, x2)

        elif delta == 0:
            return -self.b / self.a

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return (x1, x2)
```

Secvență de cod 4.1: Implementarea clasei EquationSolver și a mutanților

4.4 Teste pentru implementarea originală și pentru mutanți

Pentru validarea implementării originale și pentru verificarea faptului că mutanții sunt detectați, au fost definite teste unitare folosind framework-ul `unittest`.

```
class TestEquationSolverMutationAdequacy(unittest.TestCase):

    def test_invalid_types(self):
        with self.assertRaises(ValueError):
            EquationSolver("text", 2, 1).solve()

    def test_a_zero(self):
        with self.assertRaises(ValueError):
            EquationSolver(0, 2, 1).solve()

    def test_delta_negative(self):
        result = EquationSolver(1, 1, 1).solve()
        self.assertIsInstance(result, tuple)
        self.assertEqual(len(result), 2)
        self.assertIsInstance(result[0], complex)
        self.assertIsInstance(result[1], complex)

    def test_delta_zero(self):
        result = EquationSolver(1, 2, 1).solve()
        self.assertEqual(result, -1.0)

    def test_delta_positive(self):
        result = EquationSolver(1, 5, 6).solve()
        self.assertEqual(result, (-2.0, -3.0))

    def test_mutant_aor_is_killed(self):
        original = EquationSolver(1, 5, 6).solve()
        mutant = EquationSolver_Mutant_AOR(1, 5, 6).solve()
        self.assertNotEqual(original, mutant)

    def test_mutant_ror_is_killed(self):
        original = EquationSolver(1, 2, 1).solve()
        mutant = EquationSolver_Mutant_ROR(1, 2, 1).solve()
        self.assertNotEqual(original, mutant)

    def test_mutant_std1_is_killed(self):
        original = EquationSolver(1, 1, 1).solve()
        mutant = EquationSolver_Mutant_STD1(1, 1, 1).solve()
        self.assertNotEqual(original, mutant)

    def test_mutant_std2_is_killed(self):
        original = EquationSolver(1, 2, 1).solve()
        mutant = EquationSolver_Mutant_STD2(1, 2, 1).solve()
        self.assertNotEqual(original, mutant)

    def test_mutant_lcr_is_killed(self):
        with self.assertRaises(Exception):
            EquationSolver_Mutant_LCR("text", 2, 1).solve()


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Secvență de cod 4.2: Teste unitare pentru EquationSolver și pentru mutanți

4.4.1 Strong vs Weak Mutation

Un test omoară un mutant dacă diferența dintre programul original și mutant este observabilă.

- **Weak mutation**: diferența apare doar în starea internă a programului.
- **Strong mutation**: diferența se propagă până la ieșire și este observabilă în rezultatul final.

În cadrul acestei lucrări se utilizează strong mutation, deoarece verificarea rezultatelor finale garantează detectarea efectivă a erorilor.

4.5 Analiza mutanților

Pentru a evalua eficiența setului de teste, fiecare mutant este executat separat împreună cu testele definite pentru clasa `EquationSolver`. Rezultatele obținute sunt sintetizate în Tabela 4.2.

Tabela 4.2: Rezultatul execuției testelor asupra mutanților

Mutant	Status	Motiv
AOR	Ucis	Testele pentru ecuații cu $\Delta > 0$ sau $\Delta < 0$ detectează imediat calculul eronat al discriminantului.
ROR	Ucis	Este detectat prin testul explicit pentru cazul de frontieră $\Delta = 0$.
LCR	Ucis	Testele pentru tipuri invalide detectează lipsa validării corecte a coeficienților.
STD-1	Ucis	Testele pentru cazul $\Delta < 0$ detectează faptul că metoda nu mai returnează rădăcinile complexe.
STD-2	Ucis	Este detectat prin testul care verifică exact valoarea rădăcinii duble pentru cazul $\Delta = 0$.

În cazul mutantului de tip ROR, comportamentul programului este modificat doar pentru valoarea de frontieră $\Delta = 0$. Dacă setul de teste conține un caz care verifică explicit această situație, mutantul este detectat și considerat ucis. În absența unui astfel de test, mutantul poate supraviețui, deoarece pentru valori negative ale discriminantului condiția $\Delta \leq 0$ rămâne adevărată și programul se comportă similar cu versiunea originală.

4.5.1 Condiții pentru detectarea mutanților

Pentru ca un test să detecteze un mutant, trebuie îndeplinite trei condiții:

- **Reachability**: instrucțiunea modificată trebuie să fie executată.
- **State infection**: modificarea trebuie să afecteze starea internă a programului.
- **State propagation**: diferența trebuie să se propage până la ieșire.

Dacă una dintre aceste condiții nu este îndeplinită, mutantul poate supraviețui, chiar dacă există o eroare în cod.

4.6 Rezultatele rulării testelor

Pentru a valida comportamentul implementării originale și pentru a verifica dacă mutanții sunt detectați, testele au fost rulate folosind framework-ul `unittest` din Python. În cazul implementării corecte, toate testele definite pentru clasa `EquationSolver` trec cu succes.

Un exemplu de rezultat obținut la rularea testelor este prezentat mai jos:

```
test_invalid_types ... ok
test_a_zero ... ok
test_delta_negative ... ok
test_delta_zero ... ok
test_delta_positive ... ok
test_mutant_aor_is_killed ... ok
test_mutant_ror_is_killed ... ok
test_mutant_std1_is_killed ... ok
test_mutant_std2_is_killed ... ok
test_mutant_lcr_is_killed ... ok

----------------------------------------------------------------------
Ran 10 tests in 0.001s

OK
```

Rezultatele obținute confirmă faptul că testele definite sunt suficiente pentru a detecta modificările introduse prin operatorii de mutație analizați.

4.6.1 Mutanți echivalenți

Un mutant este considerat echivalent dacă produce aceleași rezultate ca programul original pentru orice date de intrare.

Acești mutanți nu pot fi detectați prin teste, deoarece nu există diferențe observabile între comportamentul lor și cel al programului original.

Determinarea automată a mutanților echivalenți este, în general, o problemă nedecidabilă, motiv pentru care identificarea acestora se realizează, în practică, prin analiză manuală.

4.7 Concluzii

Analiza pe mutanți evidențiază faptul că un set de teste bun trebuie să acopere nu doar toate ramurile de execuție, ci și valorile de frontieră și rezultatele numerice exacte returnate de program.

Pentru a evalua eficiența setului de teste se poate calcula scorul de mutație (Mutation Score), definit ca raportul dintre numărul de mutanți uciși și numărul total de mutanți generați:

$$MS = \frac{M_{killed}}{M_{total}}$$

unde M_{killed} reprezintă numărul mutanților detectați de teste, iar M_{total} reprezintă numărul total de mutanți generați.

O formulare echivalentă utilizată în literatura de specialitate este:

$$MS = \frac{D}{L + D}$$

unde:

- D reprezintă numărul mutanților uciși;
- L reprezintă numărul mutanților nedetectați (live mutants).

În cazul analizat:

$$MS = \frac{5}{5} = 100\%$$

Acest rezultat arată că setul de teste este capabil să detecteze toți mutanții considerați în experiment.

În concluzie, analiza realizată demonstrează că setul de teste implementat pentru clasa `EquationSolver` este capabil să detecteze majoritatea modificărilor introduse în cod prin operatorii de mutație. Prin combinarea testării funcționale, a testării structurale și a testării pe mutanți, se obține o validare mai riguroasă a corectitudinii și robusteții algoritmului.

De asemenea, rezultatele obținute sunt în concordanță cu ipoteza competent programmer hypothesis, conform căreia erorile introduse în program sunt, de regulă, mici variații față de implementarea corectă și pot fi detectate prin mutații simple.

În plus, efectul de cuplare (coupling effect) sugerează că testele capabile să detecteze mutanți de ordin 1 sunt, în general, suficiente pentru a detecta și erori mai complexe.

Rezultatele obținute sunt în concordanță cu ipoteza competent programmer hypothesis, conform căreia erorile sunt, în general, mici deviații de la implementarea corectă.

De asemenea, efectul de cuplare (coupling effect) sugerează că testele care detectează mutanți simpli sunt suficiente pentru a detecta și erori mai complexe.